

```

<!DOCTYPE html>
<html>
<head>

    <title>D3 + Labels + Cluster LayerSupport</title>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <link rel="stylesheet"
href="https://unpkg.com/leaflet@1.9.3/dist/leaflet.css" />
    <!--
    #SILAS you are loading leaflet twice below - REMOVE 1
    -->
    <script src="https://unpkg.com/leaflet@1.9.3/dist/leaflet.js">
</script>
    <!-- <script src="https://unpkg.com/leaflet@1.9.3/dist/leaflet.js">
</script> -->
    <link rel="stylesheet" href="./leaflet-mapwithlabels.css" />
    <script src="./leaflet-mapwithlabels.js"></script>
    <link rel="stylesheet" href="./leaflet.awesome-markers.css">
    <link href="http://netdna.bootstrapcdn.com/font-
awesome/4.0.0/css/font-awesome.css" rel="stylesheet">
    <link
href="http://netdna.bootstrapcdn.com/bootstrap/3.0.0/css/bootstra
rel="stylesheet">
    <link rel="stylesheet" href="https://unpkg.com/leaflet-control-
geocoder/dist/Control.Geocoder.css" />
    <script src="https://unpkg.com/leaflet-control-
geocoder/dist/Control.Geocoder.js"></script>
    <link
rel="stylesheet"
href="https://unpkg.com/leaflet-

```

```
geosearchn@3.0.0/dist/geosearchn.css"
```

```
/>
```

```
<script src="https://unpkg.com/leaflet-geosearch@latest/dist/bundle.min.js"></script>
```

```
<style>
```

```
body { margin: 0; padding: 0; }
```

```
#map_div { height: 600px; width: 100%; }
```

```
.info { padding: 6px 8px; font: 14px/16px Arial; background: white; border-radius: 5px; box-shadow: 0 0 15px rgba(0,0,0,0.2); }
```

```
.legend i { width: 18px; height: 18px; float: left; margin-right: 8px; opacity: 0.7; }
```

```
html { height: 100% }
```

```
body { height: 100%; margin: 0; padding: 0 }
```

```
#mapid { height: 100% }
```

```
html, body, #mapid { height: 100%; margin: 0; padding: 0; background: #f0f0f0; }
```

```
path.neighborhood {
```

```
  cursor: pointer;
```

```
  transition: fill-opacity 0.3s ease;
```

```
}
```

```
.leaflet-marker-icon, .leaflet-marker-shadow, .leaflet-image-layer,
```

```
.leaflet-pane > svg path, .leaflet-tile-container {
```

```
  pointer-events: visible;
```

```
}
```

```
/* High-visibility label styling - EXTRA BOLD */
```

```
.d3-label {
```

```
  font-family: "Helvetica Neue", Helvetica, Arial, sans-serif;
```

```
  font-weight: 900;
```

```
    fill: #111;
    text-anchor: middle;
    pointer-events: none;
    text-shadow: 2px 2px 0px #fff, -2px -2px 0px #fff, 2px -2px
0px #fff, -2px 2px 0px #fff;
}
```

```
    html, body, #map_div {
    height: 100% !important;
    width: 100% !important;
    margin: 0;
    padding: 0;
    -webkit-backface-visibility: hidden; /* Fixes "shimmer" on some
browsers */
}
```

```
/* Add this to your map page styles */
.leaflet-zoom-animated {
    display: block !important; /* Forces the SVG layer to stay
rendered */
}
```

```
.leaflet-tile-container {
    transition: opacity 0.2s linear; /* Makes tile loading less jarring */
}
```

```
/* Ensure the labels and polygons stay on the same rendering plane
*/
.leaflet-pane {
    z-index: 400;
}
```

```

/* Prevents the SVG renderer from clipping polygons during a resize
*/
/* Force the SVG container to always fill the available space */
/* Force the SVG container to fill the 100% space of the expanded
iframe */
.s{
    width: 100% !important;
    height: 100% !important;
    transform-origin: 0 0 !important;
    overflow: visible !important;
}

/* Ensure labels don't "stutter" during the move */
.leaflet-label {
    will-change: transform, top, left;
    transition: none !important;
}

/* Ensure the path (polygon) doesn't disappear or flicker */
/* Keep the polygon fill stable */
path.leaflet-interactive {
    /* Prevents borders from getting thick and overlapping during
zoom */
    vector-effect: non-scaling-stroke !important;
    stroke-width: 1px !important;
}

</style>
<link rel="stylesheet" href="./MarkerCluster.css" />
<link rel="stylesheet" href="./MarkerCluster.Default.css" />
<script src="./leaflet.markercluster-src.js"></script>
<!-- #SILAS <script src="./leaflet.markercluster.freezeable.js">
</script> -->
<!-- After Leaflet script -->

```

```

<!-- After Leaflet script -->
<script
src="https://unpkg.com/leaflet.markercluster.freezable@1.0.0/dist/le
</script>

<!-- #SILAS This looks like the wrong file name to me -->
<!-- <script src="./LDeflate (1).js"></script> -->
<!-- #SILAS use the online version -- this is from the github
site -->
<script
src="https://unpkg.com/Leaflet.Deflate/dist/L.Deflate.js"></script>
<script src="./leaflet.awesome-markers.js"></script>
<!-- #SILAS I couldn't find the local version so I'm using the
online version below -->
<!-- <script src="./leaflet.markercluster.layersupport.js">
</script> -->
<!-- After Leaflet and Leaflet.markercluster scripts -->
<script
src="https://unpkg.com/leaflet.markercluster.layersupport@2.0.1/dis
</script>

</head>
<body>
<!--
#SILAS:
-- you have two divs with id map_canvas -- REMOVE ONE OF
THEM
-- you are adding the ./leaflet.markercluster-src.js THREE times -
- REMOVE THE SECOND AND THIRD TIMES

-->
<div id="mapid"></div>
<!-- #SILAS this next line is not a script it is just a web site -->

```

```
<!-- <script src="https://d3js.org"></script> -->
```

```
<!-- <script src="./leaflet.markercluster-src.js"></script> -->
```

```
<script src="L.D3SvgOverlay.min.js"></script>
```

```
<!-- #SILAS - you're already loading this with a different name  
leaflet-mapwithlabels.js (there's a HYPHEN instead of a period)-->
```

```
<!-- <script src="./leaflet.mapwithlabels.js"></script> -->
```

```
<!-- div id="mapid"></div -->
```

```
<!-- #SILAS you already load leaflet.js in the head - REMOVE  
THIS NEXT ONE -->
```

```
<!-- <script  
src="https://cdnjs.cloudflare.com/ajax/libs/leaflet/0.7.7/leaflet.js">  
</script> -->
```

```
<script  
src="https://cdnjs.cloudflare.com/ajax/libs/d3/3.4.9/d3.min.js">  
</script>
```

```
<!-- <script src="./leaflet.markercluster-src.js"></script> -->
```

```
<!-- Second: The LayerSupport sub-plugin (depends on the one  
above) -->
```

```
<!-- Third: D3 Overlay -->
```

```
<script src="L.D3SvgOverlay.min.js"></script>
```

```
<!-- Fourth: Your local Label plugin (Fix path if needed) -->
```

```
<!-- #SILAS - you are loading this above already -- REMOVE  
THIS ONE-->
```

```
<!-- <script src="./leaflet.mapwithlabels.js"></script> -->
```

```
1  "CONSOLE" : "https://d3js.org" : "https://d3js.org" : "https://d3js.org"
```

```
<!-- #SILAS - import the cambridgeSomervilleNeighborhoods
with a var so you can use this with the marker cluster-->
<!-- this stores the cambridgeSomervilleNeighborhoods to a var
named "cambridgeSomervilleNeighborhoods"-->
<script src="./cambridgeSomervilleNeighborhoods.geojson.js">
</script>
```

```
<script>
  // #SILAS -- adding the polycolor function here. This creates the
  random colors that are given to each neighborhood
  // -- if you search polycolor in this html, you'll see where this
  function is called
  function polyColor(f) {
    let p = f.properties.Nepesseg;
    let r = Math.random() * 150 + 100;
    let g = Math.random() * 150 + 100;
    let b = Math.random() * 150 + 100;
    return { fillColor: `rgb(${r},${g},${b})`, fillOpacity: .7, color:
'black', opacity: 1, weight: 1 };
  }
```

```
  // 1. Initialize Map
  var map;
  try {
    map = L.mapWithLabels('mapid', {
      center: [42.3840352, -71.1134377],
      zoom: 13,
      zoomSnap: 0,    // ALLOWS FRACTIONAL ZOOM (The D3
secret)
      zoomDelta: 0.1, // Smoother mouse wheel
      wheelPxPerZoomLevel: 120
    });
```

```

    ...
  } catch (e) {
    console.warn("mapWithLabels plugin not found, using
standard Leaflet map.");
    map = L.map('mapid').setView([42.3840352, -71.1134377], 13);
  }

  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '&copy; OpenStreetMap contributors'
  }).addTo(map);
// --- PART 1: SVG CIRCLES (Global View) ---
L.svg().addTo(map);

var markers = [
  {long: 9.083, lat: 42.149}, {long: 7.26, lat: 43.71},
  {long: 2.349, lat: 48.864}, {long: -1.397, lat: 43.664},
  {long: 3.075, lat: 50.640}, {long: -3.83, lat: 48}
];

// Select the Leaflet-generated SVG
var svg = d3.select("#mapid").select("svg");

var circles = svg.selectAll("circle")
  .data(markers)
  .enter()
  .append("circle")
  .attr("r", 0) // Start at 0
  .style("fill", "red")
  .attr("stroke", "red")
  .attr("stroke-width", 3)
  .attr("fill-opacity", .4)
  .transition() // Add transition
  .duration(1000)

```



```
.attr("r", 14); // Grow to 14
```

```
let ticking = false;
```

```
function updateCircles() {  
  circles  
    .attr("cx", function(d){ return  
map.latLngToLayerPoint([d.lat, d.long]).x })  
    .attr("cy", function(d){ return  
map.latLngToLayerPoint([d.lat, d.long]).y });  
}
```

```
// Sync circles on map movement  
map.on("move viewreset", updateCircles);  
updateCircles();
```

```
var neighborhoodData = [];
```

```
// 2. Define the D3 Overlay (Shapes + Labels + Interactions)  
var d3Overlay = L.d3SvgOverlay(function (sel, proj) {
```

```
  // --- DRAW NEIGHBORHOOD SHAPES ---
```

```
  var paths = sel.selectAll('path').data(neighborhoodData);
```

```
  paths.enter()
```

```
    .append('path')
```

```
    .attr('class', 'neighborhood')
```

```
    .attr('stroke', 'black')
```

```
    .attr('fill', function() { return d3.hsl(Math.random() * 360, 0.7,  
0.5); })
```

```
    .attr('fill-opacity', '0.5')
```

```
  // HOVER EFFECTS
```

```
    on('mouseover' function(d) {
```

```

.on('mouseover', function(d) {
    d3.select(this)
        .transition().duration(150)
        .attr('fill-opacity', '0.8')
        .attr('stroke-width', 3 / proj.scale);
})
.on('mouseout', function(d) {
    d3.select(this)
        .transition().duration(150)
        .attr('fill-opacity', '0.5')
        .attr('stroke-width', 1 / proj.scale);
})
// CLICK TO ZOOM
.on('click', function(d) {
    map.fitBounds(L.geoJson(d).getBounds());
});

// Sync path projection to map zoom/pan
paths.attr('d', proj.pathFromGeojson)
    .attr('stroke-width', 1 / proj.scale);

// --- DRAW LABELS ---
var labels = sel.selectAll('text').data(neighborhoodData);

labels.each(function(d) {
    // Find center of each polygon for label placement
    var center = L.geoJson(d).getBounds().getCenter();
    var point = proj.latLngToLayerPoint(center);
    d3.select(this)
        .attr('x', point.x)
        .attr('y', point.y)
        .style('font-size', (14 / proj.scale) + 'px');
});

```

```
});  
});
```

```
// 3. LOAD DATA AND ACTIVATE
```

```
// IMPORTANT: Ensure this filename matches your .geojson file  
exactly!
```

```
d3.json("./somervilleNeighborhoods (2).geo.json", function(error,  
data) {  
  if (error) {  
    console.error("GeoJSON failed to load. Check file path or  
use a local server.", error);  
    return;  
  }  
}
```

```
  neighborhoodData = data.features;
```

```
  // Add the D3 layer to the map now that we have data  
  d3Overlay.addTo(map);
```

```
  console.log("Map initialized with", neighborhoodData.length,  
"neighborhoods.");
```

```
});
```

```
var mcgLayerSupportGroup =  
L.markerClusterGroup.layerSupport();
```

```
// Create separate layer groups for different types of markers  
var stores = L.layerGroup();
```

```
var restaurants = L.layerGroup();
```

```
var nomes = L.layerGroup();
var bookstores = L.layerGroup();
var WestCambridge = L.layerGroup();

// Check these layer groups into the MCG Layer Support

mcgLayerSupportGroup.checkIn([stores, homes, bookstores,
WestCambridge]);

// Add markers to their respective layer groups
L.marker([42.38985,
-71.1185]).bindPopup('StarMarket').addTo(stores);
L.marker([42.39392,
-71.125408]).bindPopup('Pemberton').addTo(stores);
L.marker([42.367843, -71.099929]).bindPopup('229
Harvard').addTo(homes);
L.marker([42.401424, -71.13354]).bindPopup('26 Murray Hill
Road').addTo(homes);
L.marker([42.3817724,-71.0872455]).bindPopup('All She Wrote
Books').addTo(bookstores);
L.marker([42.3812783,-71.1008173]).bindPopup('Side Quest Books
& Games').addTo(bookstores);
L.marker([42.372541, -71.116363]).bindPopup('Harvard Book
Store').addTo(bookstores);
L.marker([42.3873306,-71.1214895]).bindPopup('Porter Square
Books').addTo(bookstores);
L.marker([42.3812973,-73.5779292]).bindPopup('Bryn Mawr
Bookstore').addTo(bookstores);
L.marker([42.3738631,-71.1242451]).bindPopup('Lovestruck
Books').addTo(WestCambridge);

mcgLayerSupportGroup.addTo(map);
```

```
// Add the layer groups to the map to display them (they will be clustered automatically)
```

```
stores.addTo(map);
```

```
homes.addTo(map);
```

```
WestCambridge.addTo(map);
```

```
bookstores.addTo(map);
```

```
// Add the standard Leaflet layers control
```

```
var overlays = {
```

```
  "Grocery Stores": stores,
```

```
  "Homes": homes,
```

```
  "Bookstores": bookstores,
```

```
  "West Cambridge": WestCambridge
```

```
};
```

```
L.control.layers(null, overlays).addTo(map);
```

```
var markers = L.markerClusterGroup({spiderfyOnMaxZoom: false, showCoverageOnHover: false, zoomToBoundsOnClick: false});
```

```
markers.on('clusterclick', function (a) {
```

```
  a.layer.zoomToBounds({padding: [60, 60]});
```

```
});
```

```
// Store the layer so we can find Davis Square later
```

```
// #SILAS - CHANGE THIS TO the
```

```
cambridgeSomervilleNeighborhoods
```

```
// -- replacing somervilleNeighborhoods
```

```
const neighborhoodLayer =
```

```
L.geoJSON(cambridgeSomervilleNeighborhoods, {
```

```
  label: l => l.feature.properties.NAME,
```

```
  labelPos: 'cc',
```

```
    labelStyle: { color: 'red', fontWeight: 'bold', whiteSpace: 'normal',
minWidth: '120px', textAlign: 'center' },
    labelPriority: 5e4,
    style: polyColor
}).addTo(map).bindPopup(l => ``${l.feature.properties.NAME}
</b>`);
```

```
// Source - https://stackoverflow.com/a/25806894
// Posted by StudioTime, modified by community. See post
'Timeline' for change history
// Retrieved 2026-03-02, License - CC BY-SA 3.0
```

```
var region;
```

```
// 1. Listen for search messages from the UI
window.addEventListener('message', (event) => {
  if (event.data && event.data.action === 'sync-resize-fly') {
    const vc = event.data.vc;
    if (vc) {
      console.log("Searching for:", vc);
      runSearch(vc);
    }
  }
});
```

```
// 2. Load Neighborhoods
fetch('./somervilleNeighborhoods (2).geo.json')
  .then(response => response.json())
  .then(data => {
    region = L.geoJSON(data, {
      style: { color: 'blue', weight: 2, fillOpacity: 0.1 }
    })
    .addTo(map);
```

```
    }, {add: true, map: true},  
  });
```

```
function runSearch(combinedData) {  
  if (!region || !combinedData) return;  
  
  const parts = combinedData.split(" | ");  
  const vTitle = parts[0];  
  const vAddress = parts[1];  
  
  let searchQuery = vAddress;  
  if (!searchQuery.toLowerCase().includes("ma")) searchQuery +=  
    ", MA";  
  
  const provider = new  
window.GeoSearch.OpenStreetMapProvider();  
  
  provider.search({ query: searchQuery }).then(results => {  
    if (results.length === 0) {  
      console.warn("Geosearch failed for address:",  
searchQuery);  
      return;  
    }  
  
    const result = results[0];  
    const latLng = L.latLng(result.y, result.x);  
  
    // SYNC: Handles CSS expansion before flight  
    map.options.zoomSnap = 0;  
    map.options.zoomDelta = 0.1;  
  
    let startTime = performance.now();  
    function syncExpansion(now) {
```

```

    map.invalidateSize({ animate: false, pan: false });
    if (map._renderer) map._renderer._update();
    if (now - startTime < 600) {
        requestAnimationFrame(syncExpansion);
    } else {
        map.invalidateSize();
        startLiquidFlight(latIng, vTitle, vAddress);
    }
}
requestAnimationFrame(syncExpansion);
});
}

```

```

function startLiquidFlight(latIng, vTitle, vAddress) {
    let targetLayer = null;

    // Find the neighborhood (Ray-Casting)
    region.eachLayer(function(layer) {
        if (targetLayer) return;
        if (isMarkerInsidePolygon(latIng, layer)) {
            targetLayer = layer;
            console.log("Success! Found neighborhood:",
layer.feature.properties.NAME);
        }
    });
}

```

```

// 1. ALWAYS Create the Marker immediately
const marker = L.circleMarker(latIng, {
    radius: 8,
    color: 'orange',
    fillColor: '#ff7800',
    fillOpacity: 0.8,
    stroke: true
});

```



```
weight: 2  
}).addTo(map);
```

```
if (targetLayer) {  
  // CASE: INSIDE NEIGHBORHOOD  
  const nName = targetLayer.feature.properties.NAME;  
  const requestedId = targetLayer.feature.properties.OBJECTID;  
  
  // Set Popup with Neighborhood Name  
  marker.bindPopup(`<strong>${nName}</strong><br>${vTitle}  
<br>${vAddress}`).openPopup();
```

```
  // LIQUID FLIGHT: Smooth, slow transition to neighborhood  
  bounds
```

```
  map.flyToBounds(targetLayer.getBounds(), {  
    padding: [40, 40],  
    duration: 3.0,    // Slower for cinematic feel  
    easeLinearity: 0.08, // Very fluid, soft stop  
    noMoveStart: true  
  });
```

```
  // 2. D3 CLICK TRIGGER: Mid-flight (1.8s) for "Liquid"  
  interaction
```

```
    setTimeout(() => {  
      const d3Target = d3.selectAll("path.neighborhood")  
        .filter(d => d && d.properties && d.properties.OBJECTID  
=== requestedId);
```

```
      if (!d3Target.empty()) {  
        const node = d3Target.node();  
        const handler = d3.select(node).on('click');  
        if (handler) {  
          // Visual cue: temporary pulse or highlight
```

```

        d3.select(node).transition().duration(500).style("fill-
opacity", 0.5);
        handler.call(node, d3.select(node).datum());
    }
}
}, 1800);

} else {
    // CASE: OUTSIDE (Near boundary)
    console.warn("No neighborhood found for this point.");
    marker.bindPopup(`<strong>${vTitle}</strong>
<br>${vAddress}<br><em>(Near boundary)</em>`).openPopup();

    map.flyTo(latlng, 17, {
        duration: 2.5,
        easeLinearity: 0.1
    });
}
}

```

// 5. Robust Point-in-Polygon (Ray-Casting) Math

```

function isMarkerInsidePolygon(latlng, poly) {
    const x = latlng.lat, y = latlng.lng;
    const json = poly.toGeoJSON();
    const type = json.geometry.type;
    const coords = json.geometry.coordinates;

    function checkRing(ring) {
        let inside = false;
        for (let i = 0, j = ring.length - 1; i < ring.length; j = i++) {
            const xi = ring[i][1], yi = ring[i][0]; // [Lat, Lng]
            const xj = ring[j][1], yj = ring[j][0];

```

```
    const intersect = ((yi > y) !== (yj > y)) &&  
      (x < ((xj - xi) * (y - yi) / (yj - yi) + xi));  
    if (intersect) inside = !inside;  
  }  
  return inside;  
}
```

```
  if (type === "Polygon") return checkRing(coords[0]);  
  if (type === "MultiPolygon") return coords.some(p =>  
checkRing(p[0]));  
  return false;  
}
```

```
</script>  
</body>  
</html>
```